BombLab 2026 逆向工程实验报告

参数化拆弹求解器的设计与实现

计算机系统基础（ICS / CSAPP）课程实验

课程： ICS / CSAPP
实验： BombLab 2026
日期： 2026-10-30
提交物： `defuser.c`, `report.pdf`

六道正式关卡 + Honor • 通用求解，不写死任何单个 seed 的答案

摘要

本实验围绕 BombLab 的六道正式关卡与 Honor 关，设计并实现了一个接收 64 位 `challenge` 并输出对应口令的通用求解器。程序由 `seed` 生成不同的 `challenge`，因此求解目标是从 `bomb` 的参数化汇编中恢复每个关卡的计算关系，并将该关系实现于仅允许修改的 `defuser.c`。

实验主要使用 `objdump` 获取反汇编，使用 `nm` 定位子函数与数据符号，再结合 x86-64 System V 调用约定、32 位无符号回绕、跳转表、递归、结构体布局和函数指针表等知识逐关分析。最终六关公开测试为 **60/60**，Honor 公开测试为 **10/10**；并使用 seed 43、12345、777777 等公开列表之外的 seed 进行了联合验证。

关键词：BombLab；x86-64；逆向工程；参数化求解；跳转表；递归；链表；函数指针

目录

1 引言 **4**

2 实验环境与总体方法 **4**

2.1 实验环境 4

2.2 总体逆向流程 4

3 关卡分析与求解 **5**

3.1 第一关：函数调用与三步计算 5

3.1.1 p1_compute 的参数拆分 5

3.1.2 三个步骤的汇编与公式 6

3.1.3 代码经验 6

3.2 第二关：数组、循环与递推 7

3.2.1 第一个字段 7

3.2.2 循环与递推关系 8

3.3 第三关：跳转表与分支调度 9

3.3.1 p3_dispatch 与跳转表 9

3.3.2 八个 case 的计算关系 11

3.4 第四关：递归遍历与返回过程 11

3.4.1 入口参数 12

3.4.2 p4_walk 的递归关系 13

3.5 第五关：结构体、指针与链表遍历 13

3.5.1 节点布局与静态数据 14

3.5.2 遍历与哈希关系 15

3.6 第六关：函数指针表与间接调用 15

3.6.1 调度关系 16

3.6.2 函数指针表的真实映射 16

3.6.3 六个处理函数 16

3.7 Honor：观测台旧备份 17

3.7.1 secret_gate：输入长度与偏移检查 17

3.7.2 secret_phase 的三个字段 17

3.8 关键函数用途汇总 19

4 测试与结果 **20**

4.1 构建与公开测试 20

4.2 非公开 seed 验证 20

4.3 完整运行与 Honor 联合验证 21

4.4 误差与边界检查 22

5 讨论 **23**

5.1 主要难点与经验 23

5.2 可复现性 23

5.3 AI 工具使用说明 23

5.3.1 辅助范围 23

5.3.2 引用资料 23

6 结论 **24**

A 原始实验记录摘录（尽量保留原话） **24**

A.1 第一关原始记录 24

A.2 第二关原始记录 25

A.3 第三关原始记录 25

A.4 第四到第六关原始记录 26

A.5 Honor 原始记录 27

致谢与尾声 **27**

1 引言

计算机系统基础课程中的 BombLab 以拆弹为背景，要求学生在没有源代码的条件下分析 x86-64 可执行文件，恢复每一关的输入检查逻辑。与只需要得到某一关固定密码的传统做法不同，本实验的求解器接收由 seed 派生的 64 位 challenge，因此必须给出通用计算关系，不能写死任何单个 seed 的答案。

本实验覆盖六道正式关卡和一个必做 Honor 关。六道正式关卡分别涉及函数调用、数组与循环、跳转表、递归、链表和函数指针表；Honor 关还要求结合 context 构造完整的一行文本。报告的主要工作包括：一是逐关提取关键汇编并恢复数学关系；二是将关系实现为 32 位无符号 C 运算；三是通过公开测试、非公开 seed 和完整通关记录验证通用性。

本文余下部分组织如下：第 2 节介绍实验环境与总体方法；第 3 节分析各关卡的指令流和计算关系；第 4 节给出测试与验证结果；第 5 节讨论实验中的主要问题与经验；第 6 节总结全文。原始口语化调试记录和 AI 工具使用说明放在附录中。

2 实验环境与总体方法

2.1 实验环境

实验在 WSL Ubuntu 24.04 中完成。求解器只修改 `defuser.c`，不修改 `bomb`、驱动、测试程序或接口。编译命令由 Makefile 统一调用 GCC，并开启 `-Wall -Wextra -Wpedantic -Werror`。

```
cd "/home/cysjake/26fall-cs basic/lab2/Bomblab-cys"
make
python3 public_test.py
./defuser --seed 43 --phase 1 | ./bomb --seed 43 --phase 1
```

```
cysjake@CYS-HELPER:~$ cd: home/cysjake/26fall-cs basic/lab2/Bomblab-cys"
>
> ^C
cysjake@CYS-HELPER:~$ cd: home/cysjake/26fall-cs basic/lab2/Bomblab-cys
Command 'cd:' not found, did you mean:
  command 'cdb' from deb tinycdb (0.81-1)
  command 'cdi' from deb cdo (2.3.0-1)
  command 'cdw' from deb cdw (0.8.1-3)
  command 'cd5' from deb cd5 (0.1-4)
  command 'cdp' from deb irpas (0.10-9)
  command 'cde' from deb cde (0.1+git9-g551e54d-1.2)
  command 'cdo' from deb cdo (2.3.0-1)
Try: sudo apt install <deb name>
cysjake@CYS-HELPER:~$ cd "/home/cysjake/26fall-cs basic/lab2/Bomblab-cys"
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$ make
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challenge.c cli.c -o defuser
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$ make
./defuser --seed 42 --phase 1 | ./bomb --seed 42 --phase 1
python3 public_test.py --phase 1
./defuser --seed 43 --phase 1 | ./bomb --seed 43 --phase 1
make: Nothing to be done for 'all'.
```

图 1: 在 WSL 中进入仓库、编译并开始单关验证。截图保留了实际命令历史，包括最初路径输入错误，随后使用绝对路径成功进入仓库。

2.2 总体逆向流程

1. 用 `objdump -d --no-show-raw-insn --disassemble=phase_0X` 导出目标函数；

2. 用 `nm -an bomb` 找到 `phase_0X` 调用的 `pX_*` 子函数和数据表；
3. 按照 System V AMD64 ABI 跟踪 `rdi/rsi/rdx/rcx/r8/r9` 与返回值 `eax/rax`；
4. 把汇编中的 32 位运算严格翻译为 `uint32_t` 运算；
5. 用公开测试和公开列表之外的 seed 验证，不能只凭一次运行判断正确。

本实验中的一个统一约定是：程序虽然运行在 64 位环境，绝大多数关卡的字段和中间结果都是 `uint32_t`。因此只要汇编使用 `eax/edi/esi/edx` 这类 32 位寄存器，C 代码就必须用 `uint32_t` 承接，并允许自然回绕。

3 关卡分析与求解

3.1 第一关：函数调用与三步计算

分析过程：我先运行 `objdump -d --no-show-raw-insn --disassemble=phase_01 ./bomb`，在 `phase_01` 里发现 `p1_compute` 调用。这个函数先把 64 位 `challenge` 拆成高 32 位、右移 16 位后的低 32 位和低 32 位；接着按 `step1`、`step0`、`step2` 的顺序调用三个子函数。第一次调用时 `%rdi` 是 H、`%rsi` 是 M、`%rdx` 是 challenge；第二次调用时 `%esi` 改成 H、`%edi` 改成 L，`%rcx` 保存 step1 的返回值；第三次调用时 `%rdi` 是 step0 返回值、`%rsi` 是 step1 返回值。我沿着这些寄存器变化，把三步计算关系串了起来。

我先运行：

```
objdump -d --no-show-raw-insn --disassemble=phase_01 ./bomb
```

在 `phase_01` 中发现一个关键调用：

```
4020d8: mov    %r12,%rdi
4020db: call   402050 <p1_compute>
4020e0: cmp    %eax,-0x14(%rbp)
4020e3: sete   %al
```

这说明本关只解析一个整数，真正的合法值由 `p1_compute(challenge)` 返回，输入必须与这个返回值相等。

3.1.1 p1_compute 的参数拆分

`p1_compute` 先把 64 位 `challenge` 拆成三部分：高 32 位、右移 16 位后取低 32 位、低 32 位。记：

```
H = (uint32_t)(challenge >> 32)
L = (uint32_t)challenge
M = (uint32_t)(challenge >> 16)
```

按照汇编中的调用顺序，三个步骤是 p1_step1 → p1_step0 → p1_step2：

```
s1 = p1_step1(H, M);
s0 = p1_step0(L, H);
result = p1_step2(s0, s1);
```

3.1.2 三个步骤的汇编与公式

p1_step1：

```
lea    (%rdi,%rsi,8),%eax   // eax = H + 8 * M
xor    $0x5da7,%eax         // eax ^= 0x5DA7
```

```
s1 = (H + 8u * M) ^ 0x5DA7u;
```

p1_step0：

```
lea    0xd6da(%rsi),%eax    // eax = H + 0xD6DA
xor    %edi,%eax            // eax ^= L
```

```
s0 = (H + 0xD6DAu) ^ L;
```

p1_step2：

```
lea    (%rdi,%rdi,4),%eax   // eax = 5 * s0
add    $0x4be7,%esi         // esi = s1 + 0x4BE7
xor    %esi,%eax            // eax = (5 * s0) ^ (s1 + 0x4BE7)
```

```
answer = (5u * s0) ^ (s1 + 0x4BE7u);
```

3.1.3 代码经验

这一关的关键在调用顺序和寄存器复用。第一次调用时 %rdi = H、%rsi = M；第二次调用前，%esi 被改成 H、%edi 被改成 L；第三次调用时 %ecx 保存了 s1，因此 p1_step2 的第二个参数来自 %rcx。如果把调用顺序写反，结果一定错误。最终 count = 1。



图 2: 第一关 seed 42 的 challenge 与 `Phase 1 defused.`，下方同时显示公开测试全部通过。

3.2 第二关：数组、循环与递推

分析过程：我先注意到 4020fd 把 6 存在 `%esi`，因此这一关有 6 个输入。建立栈帧后，六个整数依次放在 `rbp-0x40` 到 `rbp-0x2c`。40215d 把 a[0] 取出，与 challenge 右移 36 位后的低 8 位异或 2 比较。进入循环后，40217b 的 `lea (%rsi,%rsi,2),%esi` 把当前字段变成三倍，402181 的 `sub %edx,%esi` 再减去循环下标；402183 把 challenge 按 ecx 右移后取低四位，402186 加 5，最后与前面的三倍值异或。`%ecx` 从 10 开始，每轮加 2，对应移位量 10、12、14、16、18。我根据这些指令写出 `a[i]` 的递推关系。

`phase_02` 开头把 `%esi` 设置为 6，并调用 `valid_u32_fields(input, 6)`；随后使用格式串 `"%u %u %u %u %u %u"` 解析 6 个整数。输入数组在栈上连续分布：

字段	地址
a[0]	rbp-0x40
a[1]	rbp-0x3c
a[2]	rbp-0x38
a[3]	rbp-0x34
a[4]	rbp-0x30
a[5]	rbp-0x2c

3.2.1 第一个字段

```
mov    %r13,%rax
```

```
mov    -0x40(%rbp),%esi
shr    $0x24,%rax
movzbl %al,%eax
xor    $0x2,%eax
cmp    %eax,%esi
```

右移 36 位再取低 8 位，即取 challenge 的 bits 36..43，然后异或 2：

```
a[0] = ((uint32_t)(challenge >> 36) & 0xFFu) ^ 2u;
```

3.2.2 循环与递推关系

```
mov    $0xa,%ecx        // 初始移位量 10
mov    $0x1,%edx        // i = 1
loop:
mov    %r13,%rax
lea    (%rsi,%rsi,2),%esi   // esi = 3 * 上一个字段
shr    %cl,%rax
sub    %edx,%esi            // esi -= i
and    $0xf,%eax
add    $0x5,%eax
xor    %esi,%eax
mov    -0x40(%rbp,%rdx,4),%esi
cmp    %eax,%esi
jne    fail
add    $0x1,%rdx
add    $0x2,%ecx
cmp    $0x6,%rdx
jne    loop
```

循环每一轮开头都会重新把 esi 设为上一个字段，因此递推项使用 a[i-1]。我据此写出：

```
a[i] = (((challenge >> (10 + 2*(i-1))) & 0xFu) + 5u)
^ (3u * a[i-1] - (uint32_t)i);    i = 1..5
```

其中 edx 同时承担循环下标和减法操作数：第 1 轮减 1，第 2 轮减 2，直到第 5 轮减 5；ecx 则从 10 开始，每轮加 2，对应移位量 10、12、14、16、18。所有加减乘异或均为 32 位无符号回绕。最终 count = 6，按 a[0] 到 a[5] 的顺序写入答案。

```
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$ objdump -d --no-show-raw-insn --disassemble=phase_02 ./bomb > phase_02.S
nm -an bomb | grep -E 'phase_02|p2_'
00000000004020f0 T phase_02
00000000004056e0 r p2_agent_urgent
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$ ./defuser --seed 12345 --phase 1 | ./bomb --seed 12345 --phase 1^C
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$ ./defuser --seed 12345 --phase 1 | ./bomb --seed 12345 --phase 1
```

图 3: 第二关的分析命令：导出 phase_02 并查找 p2_ 相关符号。

```
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$ python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$
```

图 4: 第二关公开测试 10/10。这里也验证了递推式必须使用 a[i-1] 和随轮次变化的减法量。

3.3 第三关：跳转表与分支调度

分析过程：我同样先确认输入个数，`phase_03` 解析 2 个整数。4022fc 把 ctx 的低 32 位复制到 `%edi`，`and edi,7` 得到 selector = ctx & 7。402304 比较第一个输入与 selector，只有相等才继续。接着 402309 把 ctx 复制成两份，一份右移 3，一份右移 32，再异或得到 x，传给 `p3_dispatch`。`p3_dispatch` 根据 selector 查 0x405820 的表项跳到 `p3_case_*`，返回值再与第二个输入比较。我查看表项后得到 selector 0 到 7 对应 case 0、7、5、2、1、4、6、3。

同样先用 objdump 查看 phase_03，发现它解析 2 个整数，并把第一个输入与 challenge & 7 比较；只有相等才继续调用 p3_dispatch。设：

```
selector = (uint32_t)challenge & 7u;
x = (uint32_t)(challenge >> 3) ^ (uint32_t)(challenge >> 32);
```

则两个输入应满足：

```
values[0] = selector;
values[1] = p3_dispatch(selector, x);
```

3.3.1 p3_dispatch 与跳转表

```
p3_dispatch:
mov    eax,edi          // eax = selector
mov    edi,esi          // 第一个参数改为 x
cmp    eax,0x7
ja     invalid
notrack jmp QWORD PTR [rax*8+0x405820]
```

这里要特别注意 0x405820 跳转表的顺序：把表项和跳板都读出来后，映射关系如下：

selector	表项跳板	最终 case
0	0x402278	p3_case_0
1	0x402280	p3_case_7

2	0x402288	p3_case_5
3	0x402290	p3_case_2
4	0x402298	p3_case_1
5	0x4022a0	p3_case_4
6	0x402268	p3_case_6
7	0x402270	p3_case_3

```
00000000004022b0 <phase_03>:
  4022b0:       endbr64
  4022b4:       push   rbp
  4022b5:       mov    rbp,rsp
  4022b8:       push   r12
  4022ba:       mov    r12,rsi
  4022bd:       mov    esi,0x2
  4022c2:       push   rbx
  4022c3:       mov    rbx,rdi
  4022c6:       sub    rsp,0x10
  4022ca:       call   402c70 <valid_u32_fields>
  4022cf:       test   al,al
  4022d1:       jne    4022e0 <phase_03+0x30>
  4022d3:       xor    eax,eax
  4022d5:       add    rsp,0x10
  4022d9:       pop    rbx
  4022da:       pop    r12
  4022dc:       pop    rbp
  4022dd:       ret
  4022de:       xchg   ax,ax
  4022e0:       xor    eax,eax
  4022e2:       lea    rcx,[rbp-0x14]
  4022e6:       lea    rdx,[rbp-0x18]
  4022ea:       mov    esi,0x403186
  4022ef:       mov    rdi,rbx
  4022f2:       call   4011e0 <__isoc99_sscanf@plt>
  4022f7:       cmp    eax,0x2
  4022fa:       jne    4022d3 <phase_03+0x23>
report/q3_intel.txt
```

图 5: 第三关 phase_03 的关键汇编：解析两个整数并比较 selector。

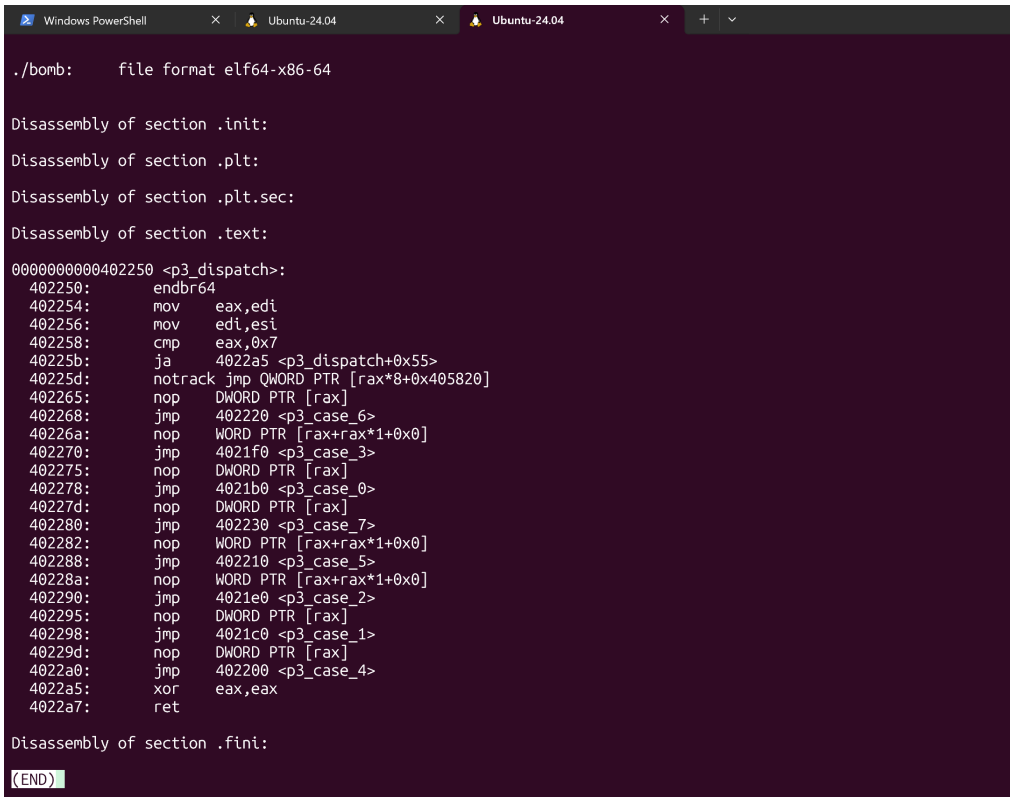


图 6: p3_dispatch 的跳转表与各跳板。跳板地址并不等于 case 编号，需要逐项对应。

3.3.2 八个 case 的计算关系

把 selector 映射到实际 case 后，得到下面的完整关系。所有运算都是 32 位无符号运算。

selector	实际 case	计算关系
0	p3_case_0	3*x + 0x20F4
1	p3_case_7	(4*x) ^ 0x5A60
2	p3_case_5	5*x + 0xA648
3	p3_case_2	(x ^ 0xA447) + (x >> 1)
4	p3_case_1	(4*x) ^ 0xA1CD
5	p3_case_4	5*x + 0x15AE
6	p3_case_6	(x ^ 0x4AAC) + (x >> 1)
7	p3_case_3	5*x + 0x3354

第一版代码把 selector 直接当成 case 编号，结果只偶然通过了个别测试。原因是跳转表本身经过了重排。最终代码显式使用上表映射，count = 2。

3.4 第四关：递归遍历与返回过程

分析过程：我在 `p4_walk` 里看到四个参数分别由 `rdi`、`esi`、`edx`、`rcx` 传入：target、lo、hi、key。函数先算 mid = lo + ((hi-lo)»1)，如果 target 等于 mid 就返回 (mid+0x29) 与 key 的异或；否

则用 mid+key 组成 mixed，并按照 target 与 mid 的大小关系递归到左区间或右区间。递归返回后再做 3*child 与 mixed 的异或，并按分支异或 0x1a 或 1。我看到这里确认它是一个带后序组合的二分递归。

在开始第四关之前，为了同时观察递归、链表和函数指针，我把第 4～6 关的 phase 函数及其子函数统一导出：

```
for f in phase_04 p4_walk phase_05 p5_traverse \
         phase_06 p6_dispatch p6_op_0 p6_op_1 p6_op_2 \
         p6_op_3 p6_op_4 p6_op_5 valid_u32_fields; do
  objdump -d -M intel --no-show-raw-insn --disassemble="$f" ./bomb
done > report/q456_asm_all.txt
```

```
cysjake@CYS-HELPER:~$ cd "/home/cysjake/26fall-cs basic/lab2/Bomblab-cys"
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$ cd "/home/cysjake/26fall-cs basic/lab2/Bomblab-cys"

declare -A funcs

funcs[4]="phase_04 p4_walk valid_u32_fields"

funcs[5]="phase_05 p5_traverse valid_u32_fields"

funcs[6]="phase_06 p6_dispatch p6_op_0 p6_op_1 p6_op_2 p6_op_3 p6_op_4 p6_op_5 valid_u32_fields"

for n in 4 5 6; do
    out="report/q${n}_asm.txt"
    : > "$out"

    for f in ${funcs[$n]}; do
        printf '\n\n===== FUNCTION: %s =====\n\n' "$f" >> "$out"
        objdump -d -M intel --no-show-raw-insn \
            --disassemble="$f" ./bomb >> "$out"
    done
done
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$
```

图 7: 批量导出第 4～6 关的 phase 与子函数汇编。后续递归、链表和函数指针分析都以这些导出结果为依据。

3.4.1 入口参数

```
mov    rcx,r12
mov    rax,r12
mov    edi,r12d
xor    esi,esi
shr    rax,0x20
shr    rcx,0x7
and    edi,0x7f
mov    edx,0x7f
xor    ecx,eax
call   p4_walk
cmp    DWORD PTR [rbp-0x14],eax
```

```
target = (uint32_t)challenge & 0x7F;
key = (uint32_t)(challenge >> 7) ^ (uint32_t)(challenge >> 32);
answer = p4_walk(target, 0, 0x7F, key);
```

按照 ABI，p4_walk 的四个参数依次是 target、lo、hi、key。

3.4.2 p4_walk 的递归关系

```
mov    eax,edi          // target
mov    edi,edx          // hi
sub    edi,esi          // hi - lo
shr    edi,1
add    edi,esi          // mid = lo + (hi-lo)/2
cmp    edi,eax
je     base
lea    ebx,[rdi+rcx]    // mixed = mid + key
cmp    eax,edi
jb     lower
call   p4_walk(target, mid+1, hi, key)
lea    eax,[rax+rax*2]  // 3 * child
xor    eax,ebx
xor    eax,0x1a
ret
```

```
mid = lo + ((hi - lo) >> 1);

if (target == mid)
return (mid + 0x29u) ^ key;

mixed = mid + key;
if (target > mid)
return (3u * child) ^ mixed ^ 0x1Au;
else
return (3u * child) ^ mixed ^ 0x01u;
```

这里的关键是递归返回后还要做一次后序组合：先乘 3，再异或 mixed，最后根据左右分支异或不同常量。递归深度很小，按二分区间最多约 7 层；count = 1。

3.5 第五关：结构体、指针与链表遍历

分析过程：`phase_05` 输入两个整数。`p5_traverse` 从 0x405b60 的头指针表中取 challenge 低 3 位对应的节点，读取节点 +8 的 value 和 +12 的 tag，用 challenge»6、challenge»32、tag、0x5d71 做初始哈希并加上 value。之后根据 (challenge»3) & 3 沿 next 走 0 到 3 步，每一步异或 tag、加上 value。函数把最后访问节点的 value 写回，主函数再与第二个输入比较，所以答案由返回值和写回值两部分组成。

phase_05 解析 2 个整数，并调用 p5_traverse：

```
lea    rsi,[rbp-0x14]
mov    rdi,r12
call   p5_traverse
mov    edx,eax
cmp    DWORD PTR [rbp-0x1c],edx
jne    fail
mov    eax,DWORD PTR [rbp-0x14]
cmp    DWORD PTR [rbp-0x18],eax
sete   al
```

```
uint32_t last;
uint32_t ret = p5_traverse(challenge, &last);
values[0] = ret;
values[1] = last;
```

3.5.1 节点布局与静态数据

每个节点占 16 字节：偏移 0 是 next 指针，偏移 8 是 value，偏移 12 是 tag。链表头表在 0x405b60，节点数组在 0x405ba0。为了避免在求解器里依赖 bomb 的绝对地址，我把指针转换成节点下标。

head 下标	入口节点下标
0	6
1	0
2	4
3	7
4	11
5	5
6	8
7	3

节点下标	next	value	tag
0	4	338	33455
1	NULL	521	54800
2	1	2870	39169
3	10	2157	59279
4	7	2353	15020
5	8	1038	38146
6	0	1819	53385

7	11	1560	37953
8	3	2620	37152
9	2	881	48049
10	9	66	5272
11	5	1339	59845

3.5.2 遍历与哈希关系

```
node = heads[challenge & 7];

acc = (uint32_t)(challenge >> 6)
^ (uint32_t)(challenge >> 32)
^ node->tag
^ 0x5D71u;

last = node->value;
acc += last;

steps = ((uint32_t)challenge >> 3) & 3u;
for (i = 0; i < steps; ++i) {
node = node->next;
last = node->value;
acc ^= node->tag;
acc += last;
}
```

这里 challenge 的位段有明确分工：低 3 位选入口节点，bits 3..4 决定额外前进几步，bits 6.. 和 bits 32.. 参与初始哈希。count = 2。链表最多只走 3 步，因此静态下标表示足够安全。

3.6 第六关：函数指针表与间接调用

分析过程：`phase_06` 只输入一个整数。`p6_dispatch` 用 challenge & 7 选第一个函数，用 (challenge»3) & 7 选第二个函数，参数为 (challenge»6) 与 (challenge»32) 的异或。第一次调用的结果异或 0x2096 后再调用第二个函数，最后加 0x6b21。我查看 0x405e00 的函数指针表，确认表下标 0 到 7 对应 op0、op3、op5、op2、op4、op4、op5、op1。

phase_06 只解析 1 个整数。关键汇编：

```
shr    rdi,0x6
mov    rax,rbx
shr    rax,0x20
xor    edi,eax
mov    rax,rbx
and    ebx,0x38
```

```
and    eax,0x7
call   QWORD PTR [rax*8+0x405e00]
xor    eax,0x2096
mov    edi,eax
call   QWORD PTR [rbx+0x405e00]
add    eax,0x6b21
```

3.6.1 调度关系

```
idx1 = (uint32_t)challenge & 7u;
idx2 = ((uint32_t)challenge >> 3) & 7u;
arg1 = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);

r1 = ops[idx1](arg1);
r2 = ops[idx2](r1 ^ 0x2096u);
answer = r2 + 0x6B21u;
```

注意 and ebx,0x38 得到的是函数指针表的字节偏移；因为表项大小为 8 字节，所以实际第二下标是 (challenge >> 3) & 7。

3.6.2 函数指针表的真实映射

函数指针表位于 0x405e00，实际映射如下：

表下标	实际函数
0	p6_op_0
1	p6_op_3
2	p6_op_5
3	p6_op_2
4	p6_op_4
5	p6_op_4
6	p6_op_5
7	p6_op_1

3.6.3 六个处理函数

```
op0(x) = (2u*x + 0x845Eu) ^ (7u*x);
op1(x) = ((x >> 3) ^ x) + 0xAE7Au;
op2(x) = ror32(x, 7) = (x >> 7) | (x << 25);
op3(x) = (x + 0x476Fu) ^ (x >> 5);
op4(x) = rol32(x, 5) ^ 0x7FAEu = ((x << 5) | (x >> 27)) ^ 0x7FAEu;
op5(x) = (x ^ 0xCD7Fu) + 8u*x;
```

最终 count = 1。实现时使用一个 p6_apply(index, x)，在内部按上表进行二次映射，避免把表下标误当成 op 编号。

3.7 Honor：观测台旧备份

分析过程：`secret_gate` 先用 strlen 检查长度在 36 到 163 之间，然后把整行复制到缓冲区。比较 DWORD [rbp-0xb0] 与 0x4e45504f，我按小端读出它是 OPEN，位置在缓冲区偏移 0x20，也就是第 32 字节。接着 `secret_phase` 从 [rbp-0xac] 即偏移 0x24 开始解析三个整数。第一个数是 context & 3；后面的 b、c 由 challenge 和 context 的高低位、`secret_mix` 以及 0xe53、0xc168 等常量组合而成。输出时我在前面放 32 字节前缀，再放 OPEN 和三个整数。

Honor 关属于必做部分。它使用额外的 context 参数，并要求构造完整的一行文本。

3.7.1 secret_gate：输入长度与偏移检查

```
call   strlen
sub    rax,0x24
cmp    rax,0x7f
jbe    accept
```

这里使用无符号减法同时检查上下界：len - 36 <= 127，因此长度为 36～163 字节。随后：

```
call   secret_copy
cmp    DWORD PTR [rbp-0xb0],0x4e45504f
jne    fail
lea    rdi,[rbp-0xac]
call   secret_phase
```

输入缓冲区起点是 rbp-0xd0。所以 rbp-0xb0 是缓冲区偏移 0x20，即第 32 字节；rbp-0xac 是偏移 0x24，即第 36 字节。也就是说，第 32 字节处必须是 OPEN，而三个整数从第 36 字节开始解析。

调试经验：

正确格式是:32 字节前缀 + OPEN + 空格 + 三个整数。例如 `00000000000000000000000000000000OPEN 1 123456789 987654321`。

3.7.2 secret_phase 的三个字段

secret_phase 从偏移 36 开始解析 a b c 三个 uint32_t。设：

```
a = (uint32_t)context & 3u;
x = (uint32_t)(challenge >> 32) ^ (uint32_t)challenge;
y = (uint32_t)(context >> 32) ^ (uint32_t)context;
```

```
r9 = (a * 0xE53u) ^ (uint32_t)(context >> 16);

mix(v) = ((v >> 1) ^ 0xC168u) + (v ^ 0xE53u);
b = mix(x ^ y) ^ r9;
c = b ^ (y + a + 0xC168u + (x >> 1));
```

因此输出的三个十进制整数依次是 a、b、c。所有中间量都按 32 位无符号回绕；snprintf 后必须检查返回值和缓冲区容量，成功时返回 0。

逆向提示：

```
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$ python3 public_te
st.py --secret
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$
```

图 8: Honor 公开测试 10/10。修正 32 字节偏移后，所有公开 seed 都通过。

```
[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

图 9: 六项检查全部通过后进入 Honor，并显示 Secret phase defused. 与 [SAFE]。

3.8 关键函数用途汇总

下表按模块归纳本实验读到的函数及其实际用途。公式细节已在各关正文给出，这里侧重说明它们在整条验证链中的作用。

函数	用途
`valid_u32_fields`	在 `sscanf` 前检查输入是否为指定数量的无符号十进制字段，并允许前后空白。六道正式关卡和 Honor 都用它做输入格式闸门。
`p1_compute`	把 `challenge` 拆成 H、M、L，按 `step1`、`step0`、`step2` 的顺序编排计算。
`p1_step1`	合并高 32 位与中部，计算 `(H + 8*M) ^ 0x5DA7`。
`p1_step0`	合并高 32 位与低 32 位，计算 `(H + 0xD6DA) ^ L`。
`p1_step2`	合并两路中间结果，计算 `5*s0 ^ (s1 + 0x4BE7)`，得到第一关口令。
`p3_dispatch`	按照 `selector = challenge & 7` 查 0x405820 跳转表，把第二个参数 x 分派到八个 case 之一。
`p3_case_0..7`	八个算术分支，分别对 x 做三倍、四倍、五倍、异或和右移等组合；具体公式见 3.3.2 节。
`p4_walk`	以二分中点递归逼近 target；命中后返回 `(mid+0x29) ^ key`，返回途中再与 `mid+key` 和分支常量组合。
`p5_traverse`	从 8 个头节点中选入口，遍历 0 到 3 个 next，累计 value 与 tag 的哈希，并把最后节点的 value 写回。
`p6_dispatch`	从 challenge 的两个 3 位字段选择两个函数，把第一段结果异或 0x2096 后送入第二段，最后加 0x6b21。
`p6_op_0..5`	六个 32 位处理函数，分别实现乘法异或、移位异或、循环右移、循环左移等运算；表下标映射见 3.6.2 节。
`secret_gate`	检查 Honor 输入长度、复制文本、确认第 32 字节处为 OPEN，再把第 36 字节起的三个字段交给 `secret_phase`。
`secret_copy`	逐字节复制以 NUL 结尾的字符串，负责把命令行输入搬入 `secret_gate` 的局部缓冲区。
`secret_phase`	解析 a、b、c 三个字段，检查 a 与 `context & 3` 相等，并按 challenge/context 计算后两个字段。
`secret_mix`	Honor 的混合函数，计算 `((v>>1) ^ 0xC168) + (v ^ 0xE53)`。
`transcript_context`	在 bomb 主流程中处理六关的记录文本，将各关输出混合成 Honor 使用的 `context`；它是外层验证链的一部分，不属于单个正式关卡的求解函数。

4 测试与结果

4.1 构建与公开测试

```
make
python3 public_test.py --phase 1
python3 public_test.py --phase 2
python3 public_test.py --phase 3
python3 public_test.py --phase 4
python3 public_test.py --phase 5
python3 public_test.py --phase 6
python3 public_test.py --secret
```

项目	结果
正式六关公开测试	**60/60 PASS**
Honor 公开测试	**10/10 PASS**
编译	使用 -Wall -Wextra -Wpedantic -Werror，无警告

4.2 非公开 seed 验证

除公开 seed 外，还使用 seed 43、12345、777777 对六关分别验证。测试脚本直接比较 bomb 输出中的 Phase x defused.。记录如下：

```
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$ for p in 1 2 3 4
5 6; do
    for s in 43 12345 777777; do
        output=$(./defuser --seed "$s" --phase "$p" \
                 | ./bomb --seed "$s" --phase "$p" 2>&1)

        if printf '%s\n' "$output" | grep -q "Phase $p defused."; then
            echo "phase $p seed $s: PASS"
        else
            echo "phase $p seed $s: FAIL"
            printf '%s\n' "$output" | tail -n 8
        fi
    done
done | tee report/test_logs/nonpublic_phases_1_6.txt
phase 1 seed 43: PASS
phase 1 seed 12345: PASS
phase 1 seed 777777: PASS
phase 2 seed 43: PASS
phase 2 seed 12345: PASS
phase 2 seed 777777: PASS
phase 3 seed 43: PASS
phase 3 seed 12345: PASS
phase 3 seed 777777: PASS
phase 4 seed 43: PASS
phase 4 seed 12345: PASS
phase 4 seed 777777: PASS
phase 5 seed 43: PASS
phase 5 seed 12345: PASS
phase 5 seed 777777: PASS
phase 6 seed 43: PASS
phase 6 seed 12345: PASS
phase 6 seed 777777: PASS
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$
```

图 10: 六关 × 三个非公开 seed 共 18 条记录全部 PASS，说明求解器覆盖公开列表之外的输入。

4.3 完整运行与 Honor 联合验证

```
{ ./defuser --seed 43; ./defuser --secret --seed 43; } \
  | ./bomb --seed 43 2>&1 \
  | tee report/test_logs/seed43_all_defused.txt
```

```
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$ cd "/home/cysjake
/26fall-cs basic/lab2/Bomblab-cys"

mkdir -p report/test_logs
make

{ ./defuser --seed 43; ./defuser --secret --seed 43; } \
  | ./bomb --seed 43 2>&1 \
  | tee report/test_logs/seed43_all_defused.txt
make: Nothing to be done for 'all'.
Phase 1 challenge: 0x9cde98852e60034b
Input (unsigned decimal integers):
Phase 1 defused.
Phase 2 challenge: 0x6ec624aa85685867
Input (unsigned decimal integers):
Phase 2 defused.
Phase 3 challenge: 0xd49ffe504bfdab7f
Input (unsigned decimal integers):
Phase 3 defused.
Phase 4 challenge: 0x006ff81b72d591ff
Input (unsigned decimal integers):
Phase 4 defused.
Phase 5 challenge: 0x18e47d0995aecbfe
Input (unsigned decimal integers):
Phase 5 defused.
Phase 6 challenge: 0x523fb4dd5d162e88
Input (unsigned decimal integers):
Phase 6 defused.
Congratulations! You've defused the bomb.
Secret phase defused.
cysjake@CYS-HELPER:~/26fall-cs basic/lab2/Bomblab-cys$
```

图 11: seed 43 的完整输出：六关全部 defused，随后 Secret phase defused.。该记录属于公开 seed 列表之外的实际运行证据。

另外，使用 make test 运行完整测试链路，输出最终为 Public tests: 60/60 与 Secret public tests: 10/10，测试程序返回成功。

4.4 误差与边界检查

- 所有字段均按 uint32_t 处理，确保移位、加法、乘法和异或与汇编的 32 位寄存器语义一致；
- 第二关的循环下标 edx 同时参与减法，不能用固定常数替代；
- 第三关、第六关的查表映射必须从二进制数据中读取，不能假设顺序；
- 第五关链表以节点下标表达指针关系，且循环步数最多为 3；
- Honor 必须满足长度 36～163，并在第 32 字节处放置 OPEN。

5 讨论

5.1 主要难点与经验

本实验的主要难点在识别指令流中的结构性关系，逐条翻译 C 是后续工作。第二关中，循环下标 `%edx` 同时参与减法，容易误写成固定常数；第三关和第六关的间接跳转表都经过重排，不能把表下标直接当作 case 或 op 编号。第四关的递归必须在返回后继续组合结果；第五关需要把只读数据中的指针链表转换为下标数组；Honor 关则要求在正确位置构造 `OPEN` 标记和三个字段。上述问题均通过重新对照寄存器、数据表和测试结果得到定位。

5.2 可复现性

求解器只修改 `defuser.c`，不依赖外部文件、网络或进程调用。所有中间量都按汇编实际宽度使用 `uint32_t`，线性执行路径不依赖特定编译器优化。公开测试和非公开 seed 均使用同一套可重复命令进行验证。

5.3 AI 工具使用说明

5.3.1 辅助范围

本报告在整理与调试过程中使用了对话式 AI 编程助手 dsh + deepseek-v4.1-flash。主要用途如下：

- 解释 x86-64 汇编指令、寄存器传递和 System V AMD64 调用约定；
- 核对 32 位无符号回绕、跳转表映射、递归返回式和结构体偏移；
- 帮助定位编译错误与测试失败原因，例如第二关递推下标、第三关跳转表乱序、第六关函数指针表乱序以及 Honor 的 32 字节偏移；
- 协助整理报告结构、公式排版和截图说明。

所有关键计算关系均由本人在本地 bomb 上使用 objdump、nm 观察寄存器与数据表，并通过公开测试和公开列表之外的 seed 重新验证。AI 没有被要求生成以 seed 为索引的固定答案表，求解器也不包含任何针对单个 seed 的硬编码口令。

5.3.2 引用资料

外部引用资料：**无**。

本实验使用的材料仅为课程提供的实验说明、README.md、bomb 二进制、objdump/nm 输出以及 C 标准库和工具自身的基础用法。

6 结论

本文完成了 BombLab 2026 六道正式关卡和 Honor 关的通用求解。通过反汇编、符号定位和寄存器级分析，恢复了各关卡的计算关系，并在 `defuser.c` 中以 32 位无符号运算实现。公开测试结果为六关 60/60、Honor 10/10；seed 43、12345、777777 等非公开 seed 也全部通过。

实验表明，参数化逆向题的关键是把某一次运行的答案转化为由 challenge 决定的计算过程，同时严格控制寄存器宽度、内存布局和间接跳转映射。该过程也说明了汇编阅读、ABI 理解和系统化测试在系统程序分析中的重要性。

参考文献

本实验未引用外部文献。分析依据为课程提供的实验说明、`README.md`、个人 `bomb` 二进制及其 `objdump/nm` 输出，以及 C 标准库和工具自身的基础用法。

A 原始实验记录摘录（尽量保留原话）

下面是调试过程中留下的分析记录，我按当时的思考顺序展开，便于看出每一关的推理和验证过程。

A.1 第一关原始记录

原始记录：我先运行 objdump -d –no-show-raw-insn –disassemble=phase_01 ./bomb 得到汇编代码。

发现有个 p1_compute 的函数调用。这个函数工作流程：1. 先把 1 放到%esi，然后开辟了一个 16 字节的空间位于%rsp，这里面通过 valid_u32_fields 检查输入格式。

p1_compute 按照 step1-0-2 的顺序调用三个函数。shr 逻辑右移 32 位，放入 rax，这时候 rax 的高 32 位变为 0，eax 就相当于 x 右移 32 位的结果，设 x1 是右移 32 位的结果，存 rax；之后 rax，rdi 都是 x1；然后把 rsi 右移 16 位，这个时候被当 32 位参数读，则只读低 32 位。

第一次调用，寄存器：rdi 是 x 高 32 位；rsi 是右移 16 位取低 32 位，相当于中部；rdx 是 x。
第二次调用，寄存器：rdi 是 x 高 32 位；rsi 是 x 低 32 位；rdx 是 x；rcx 是 step1 的返回值。
第三次调用：rdi 是 step0 返回值，rsi 是 step1 返回值。

第一步，把 64 位数按不同方式取三半。lea (%rdi,%rsi,8),%eax 得 eax = 高 32 + 8 * middle；xor $0x5da7,%eax 得 eax ^= 0x5DA7。输出即 ((x1 + 8 * middle) ^ 0x5DA7) & 0xFFFFFFFF。

第二步内部：高 32 位加上 0xD6DA 然后与低 32 位异或。lea 0xd6da(%rsi),%eax 得 eax = 高 32 + 0xD6DA；xor %edi,%eax 得 eax ^= 低 32。输出即 s0 = ((H + 0xD6DA) ^ L) & 0xFFFFFFFF。

第三步内部：lea (%rdi,%rdi,4),%eax 得 eax = 5*s0；add $0x4be7,%esi 得 esi = s1 + 0x4BE7；xor %esi,%eax 得 eax = (5*s0) ^ (s1 + 0x4BE7)。输出即 result = ((5 * s0) ^ (s1 + 0x4BE7)) & 0xFFFFFFFF。

A.2 第二关原始记录

原始记录：注意到 4020fd 把 6 存在 esi，注意到此题有 6 个输入。

建立栈帧后，推入 r13，然后把第二个参数保存到 r13，然后把 esi 设置成 6，然后 r12 入栈，rdi 保存到 r12，然后 rbx 入栈，然后再加 40 字节的局部空间，从 rbp-25 到 rbp-64，用于存储 6 个数。

然后到 402154 都是在构建输出指针，使得寄存器加栈上存储的是具体数的地址：rdx = -0x40(%rbp) = &a[0]，rcx = -0x3c(%rbp) = &a[1]，r8 = -0x38(%rbp) = &a[2]，r9 = -0x34(%rbp) = &a[3]，栈上第 1 个 = -0x30(%rbp) = &a[4]，栈上第 2 个 = -0x2c(%rbp) = &a[5]。

402155: cmp $0x6,%eax；402158: jne 402115，确认要成功解析出 6 个整数。接下来检查每个数是否满足要求。已知 40215d: mov -0x40(%rbp),%esi 是 a[0] 的位置，赋值给 esi；然后对于输入，右移 36，取低八位然后重新扩展，相当于取 36-43 位，然后与 2 异或，要求输入满足 a[0] == (((ctx >> 36) & 0xFF) ^ 2)。

接下来进入循环，循环检查 a[1] 到 a[5]。40216e 把 0xa 放到 ecx，402173 把 0x1 放到 edx。由 40217b: lea (%rsi,%rsi,2),%esi 得 esi 是三倍的 a[0]。最后可解得每轮移位量由 10 开始依次加 2。

循环开头把 esi 设为上一轮字段，因此我写出 a[i] == (((ctx >> (10 + 2*(i-1))) & 0xF) + 5) ^ (3*a[i-1] - i)。

写代码后运行 python3 public_test.py –phase 2，Phase 2 seed 0 到 seed 18446744073709551615 全部 PASS，Public tests: 10/10 成功！

A.3 第三关原始记录

原始记录：同样方法，注意到输入 2 个数。首先还是一样，保存参数。接下来 4022ca 检查输入格式，确保是两个数。

然后从 4022e0 开始用 sscanf 解析两个整数：rdx 指向第一个整数 a 的地址 [rbp-0x18]，rcx 指向第二个整数 b 的地址 [rbp-0x14]，rsi 存的是格式串地址 0x403186。

4022fc 处把 phase_03 的第二个参数 ctx 的低 32 位复制到%edi 并清零%rdi 高 32 位，接着 and edi,7 得到 selector = ctx & 7，然后比较栈上的第一个输入数 a 是否等于 selector。也就是说第一个输入数必须等于 ctx & 7，它只能在 0 到 7 之间。

接下来让 rax 和 rsi 都成为 ctx 的副本，把 rax 逻辑右移 3，rsi 逻辑右移 32，最后异或。由于都取低 32 位，相当于 x = (uint32_t)(ctx >> 3) ^ (uint32_t)(ctx >> 32)。然后接下来把 selector 和 x 传给 p3_dispatch。

p3_dispatch 内部再根据 selector 分发到对应的 p3_case_0 到 p3_case_7 去计算，返回值再和第二个输入数 b 比较。

402250: endbr64；402254: mov eax,edi；402256: mov edi,esi；402258: cmp eax,0x7；40225b: ja 4022a5；40225d: notrack jmp QWORD PTR [rax*8+0x405820]。

此时前面的 selector 在 edi 里，x 在 esi 里，经操作把 x 放到 edi。实际 x 参与计算。cmp eax,0x7 是无符号跳转，如果大于 7 返回 0，但不可能。接下来跳转表就是 0x405820 加上 selector*8。

第一版代码把 selector 直接当成 case 编号，运行后只有个别测试通过。我检查 0x405820 跳转表，确认 selector 0、1、2、3、4、5、6、7 分别对应 case 0、7、5、2、1、4、6、3。

最终确认的映射是：selector 0 到 7 分别对应 case 0、7、5、2、1、4、6、3。完整公式见正文 4.2 节。

A.4 第四到第六关原始记录

第四关原始记录：asm 里有 call valid_u32_fields、sscanf(input, fmt, &x)，然后 mov rcx,r12、mov rax,r12、mov edi,r12d、xor esi,esi、shr rax,0x20、shr rcx,0x7、and edi,0x7f、mov edx,0x7f、xor ecx,eax、call p4_walk、cmp DWORD PTR [rbp-0x14],eax、sete al。其中 r12 = challenge。

所以它做的计算是：target = challenge & 0x7f; key = ((uint32_t)(challenge >> 7)) ^ ((uint32_t)(challenge >> 32)); ans = p4_walk(target, 0, 0x7f, key)。这是一个计算二分中点，然后看要查找的目标在左边还是右边的东西。

第五关原始记录：phase_05 输入两个整数。sscanf(input, fmt, &a, &b)。lea rsi,[rbp-0x14]；mov rdi,r12；call p5_traverse；mov edx,eax；cmp DWORD PTR [rbp-0x1c],edx；jne fail；mov eax,DWORD PTR [rbp-0x14]；cmp DWORD PTR [rbp-0x18],eax；sete al。

所以 phase_05 的口令两个数是：uint32_t last; uint32_t ret = p5_traverse(challenge, &last); values[0] = ret; values[1] = last;

p5_traverse 的逻辑：node = table[challenge & 7]，0x405b60 为节点指针表；last = *(uint32_t *)(node + 8)；acc = ((uint32_t)(challenge >> 6)) ^ ((uint32_t)(challenge >> 32)) ^ *(uint32_t *)(node + 12) ^ 0x5d71；acc += last。

steps = (challenge >> 3) & 3; for (i = 1; i <= steps; i++) { node = *(void **)node; last = *(uint32_t *)(node + 8); acc ^= *(uint32_t *)(node + 12); acc += last; } *out = last; return acc;

注意循环次数：edi = (challenge >> 3) & 3，若为 0 不进入循环；否则执行 edi 次额外节点遍历。最终 *out 是最后访问节点的 +8 字段，返回值是累加/异或后的哈希。

精妙之处：challenge & 7 选链表入口节点；(challenge >> 3) & 3 决定沿 next 走几步；challenge >> 6 和 challenge >> 32 参与哈希。函数同时返回一个值并写回一个值，所以 phase_05 需要两个输入整数。

第六关原始记录：phase_06 只输入一个整数，然后 mov rdi,r12；call p6_dispatch；cmp DWORD PTR [rbp-0x14],eax；sete al。所以 values[0] = p6_dispatch(challenge)。

p6_dispatch 逻辑：idx1 = challenge & 7; idx2 = (challenge & 0x38) >> 3，即 (challenge >> 3) & 7; arg1 = ((uint32_t)(challenge >> 6)) ^ ((uint32_t)(challenge >> 32)); r1 = table[idx1](arg1); arg2 = r1 ^ 0x2096; r2 = table[idx2](arg2); return r2 + 0x6b21;

给出的 p6_op_0..5 公式如下，均为 32 位运算：op0(x) = ((2*x + 0x845e) ^ (7*x)); op1(x) = ((x >> 3) ^ x) + 0xae7a; op2(x) = ror32(x, 7); op3(x) = (x + 0x476f) ^ (x >> 5); op4(x) = rol32(x, 5) ^ 0x7fae; op5(x) = (x ^ 0xcd7f) + 8*x;

精妙之处：challenge 的低 3 位选第一个函数，bits 3..5 选第二个函数，bits 6.. 和 bits 32.. 异或后作为第一个参数，第一次结果再异或 0x2096 作为第二次参数，最后加 0x6b21。这是典型的位段调度、函数表、结果链式编码。

0x405e00 的表项按 0、3、5、2、4、4、5、1 映射到 op 函数，正文 7.2 节给出最终实现。

A.5 Honor 原始记录

原始记录：1. secret_gate 在干什么。关键汇编：call strlen；sub rax,0x24；cmp rax,0x7f；jbe 402780。即要求 36 <= strlen(input) <= 36 + 0x7f = 163。这里很精妙：用无符号减法 len - 36 后和 0x7f 比较，一条判断同时完成下界和上界检查。len < 36 会下溢成巨大无符号数，直接失败。

然后 call secret_copy；cmp DWORD PTR [rbp-0xb0],0x4e45504f；jne fail。0x4e45504f 按小端是字节 4f 50 45 4e，即 OPEN。

接着 lea rdi,[rbp-0xac]；call secret_phase。secret_phase 的参数是：rdi = 缓冲区偏移 0x24（第 36 字节），rsi = challenge，rdx = context。secret_phase 先 mov esi,0x3；call valid_u32_fields；sscanf(input+4, fmt, &a, &b, &c)，所以 OPEN 后面必须恰好是 3 个无符号十进制整数，字段之间或末尾允许空白。

然后 mov ecx,r13d；and ecx,0x3；cmp DWORD PTR [rbp-0x2c],ecx；jne fail。所以第一个字段 a = context & 3。

令 x = (uint32_t)(challenge >> 32) ^ (uint32_t)challenge；y = (uint32_t)(context >> 32) ^ (uint32_t)context；a = (uint32_t)context & 3；r9 = (a * 0xe53u) ^ (uint32_t)(context >> 16)。secret_mix 的汇编是 mov eax,edi; xor edi,0xe53; shr eax,1; xor eax,0xc168; add eax,edi，即 mix(v) = ((v >> 1) ^ 0xc168u) + (v ^ 0xe53u)。

而调用 secret_mix 的输入是 mix_in = x ^ y。所以第二个字段 b = mix(mix_in) ^ r9。成功分支 shr esi,1；lea edx,[r8+rcx+0xc168]；add edx,esi；xor eax,edx；cmp eax,DWORD PTR [rbp-0x24]。此时 eax 就是上面算出的 b，esi 是 x >> 1，r8 是 y，rcx 是 a。因此第三个字段 c = b ^ (y + a + 0xc168u + (x >> 1))。所有运算都是 32 位无符号回绕。

我检查后发现公式计算正确，问题出在 Secret 的输出格式位置。secret_gate 的汇编里有 cmp DWORD PTR [rbp-0xb0],0x4e45504f 以及 lea rdi,[rbp-0xac]；call secret_phase。输入缓冲区起点是 rbp-0xd0。rbp-0xb0 是缓冲区偏移 0x20，也就是第 32 个字节，这里必须正好是 OPEN；rbp-0xac 是偏移 0x24，即第 36 字节，从第 36 字节开始才解析 a b c 三个整数。

因此最初把 OPEN 放在开头、在末尾补空格到 36 字节的做法是错的。正确格式是 32 字节前缀 + OPEN + 空格 + 三个整数。公式本身是正确的。

致谢与尾声

自己的致谢：感谢 ICS 的老师，让我累死累活搞完这玩意后能看到有趣的这一些。

结尾的彩蛋，真的很浪漫啊啊啊!!

最后的通关文案如下（以小字展开）：

> Congratulations! You have defused the bomb.
>
> [SAFE] 六项检查全部通过。新存档已启用。
>
> 你们走回街上。送货司机已经发动了车。维修铺的灯装好了，林夏跟上了你。
>
> **Honor ・观测台旧备份**
>
> 林夏准备关闭档案列表，忽然指着最下面一项：这里还有个备份。文件夹叫留给后来的人，署名是 sky 和许遥。扉页写着：交接完成后，如果你还有时间，可以到观

测台坐坐。这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，桌上的阅读器仍在等待旧程序认可的输入。城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.

[SAFE] 城市照常运行。附加档案已恢复。

阅读器亮起来。最先出现的不是代码，是两份很早的留言。

许遥：

北方那边的通知到了，我准备去。那个项目我想试很久了。以后遇到问题，不能再把椅子滑到你旁边，问一句你看这是不是有毛病。你想留下继续手里的事，我记得你为它准备了多久。我也想看看，最后能做成什么样。

走之前出去吃顿饭吧。这次别在实验室凑合了。

——sky

隔了几天，许遥在后面添了一份回复。

吃饭的时间你定。这里的东西我会留着。你在北方有了新想法，也可以回来添。我暂时不走，这边的事刚有起色，想再试一阵。第一场雪记得拍。我想看看你一直说的北方。

——许遥

后面是这些年断断续续的更新。一张北方的雪照，一次换了方向的项目，一段拖了很久才修好的代码。sky 去了北方，许遥留在这里。观测台的灯是 sky 修的，两把椅子是许遥后来搬来的。最后一张便签贴在灯座旁边：下次回来，先坐一会儿。

林夏拉开其中一把椅子。你们坐了一会儿，没有再翻记录。街上亮起今天的新灯。离开观测台时，你想起自己收藏了很久的一份项目介绍，把它放进明天的待办。

退出前，终端收到 sky 的消息：日志看到了，新存档正常。谢谢你们，剩下的维护我来接。

实验室的灯还亮着，外套仍搭在椅背上。你确认文件都保存了，合上电脑。林夏揉了揉眼睛：还活着，精神状态另说。你把椅子推回桌下：事已至此，先睡觉吧。你们关掉灯，走出了实验室。

谢谢你远道而来，与我相遇。

晚安。

——sky

最后，附上通关画面与尾声截图作为本次实验的收尾记录。

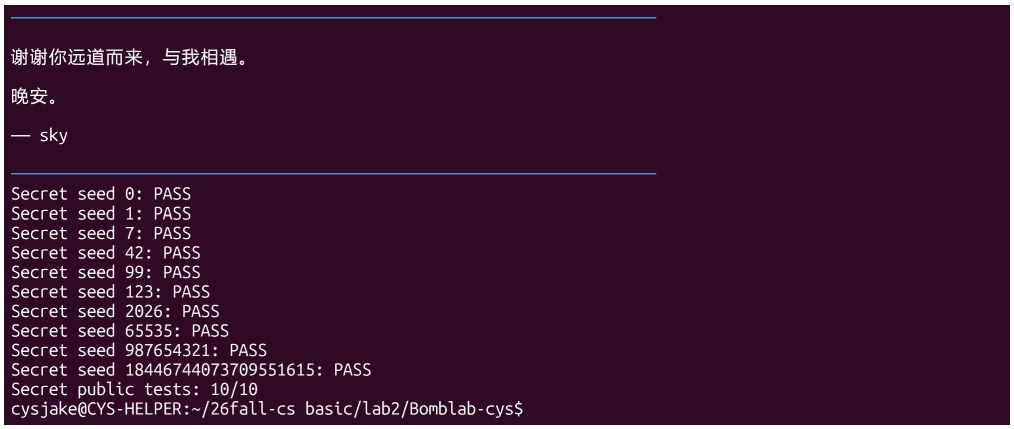


图 12: 最终通关文案、Honor 公开测试 10/10 记录与实验收束。

本报告由实际汇编分析、测试记录和终端截图整理而成。所有求解逻辑均已落到 defuser.c，没有修改 bomb、驱动、测试程序或接口。